Mira Failure Analysis (Phina)

Purpose and scope

Mira passed its 12-test evaluation in each run (Runs 2 to 5), but a passing score does not mean the system is free of failures. This document records the failures the checks missed or only partly covered, why each slipped through, and what the project does about it.

Scope: the Mira pipeline on n8n Cloud (Chat Trigger, Run Mira Core, human approval by Gmail, approved send), the code scoring on 12 tests, the risk-relevance LLM judge, and the offline judge calibration. Model: gpt-4o-mini. Failures are logged in the Known Limitations tab of the Mira Eval Results sheet (KL-1 to KL-10).

Results at a glance

Each run (Runs 2 to 5) passed 12 of 12 tests with 0 hallucinations, yet the log holds ten known limitations that the checks did not catch or only partly cover.

Run	Tests passed	Hallucinations	Avg latency (ms)	Cost per request	Cost per 100 requests	Risk judge (T3, T7)
Run 2: after orchestrator fix	12 of 12	0	3,124	$0.000319	$0.032	not run
Run 3: T11 prompt fix	12 of 12	0	2,888	$0.000318	$0.032	not run
Run 4: with risk judge	12 of 12	0	2,814	$0.000315	$0.032	2 of 2 pass, average 5
Run 5: stricter judge (rule 8)	12 of 12	0	3,472	$0.00032	$0.03	2 of 2 pass, average 5

Run 2 first passed T11 by mistake: an extra 2026-05-05 milestone sat in the date window. The check was tightened and a prompt rule added, and Run 3 fixed it. Run 4 added the risk-relevance judge. Run 5 added judge rule 8, and a fake risk (R11) scored 3 and 4 before the rule and 2 and 2 after, while real answers stayed at 5. Costs for Runs 2 to 4 are calculated from tokens, Run 5 is measured in Langfuse, and judge calls are excluded.

Failure catalogue

The log has ten entries. All ten are written up below.

ID	Test	Finding	Severity	Why the check missed it	Mitigation in place
KL-1	T1	Intermittent: seen in Run 3, absent in Runs 4 and 5 (Run 2 output truncated). The plan states the project starts on 2026-04-10 and ends 2026-08-04 over 24 weeks. April 10 is the Phase 1 end date, not a start date, and the span is about 16 weeks.	Medium (wrong date in a client-facing plan)	Score only checks that all 8 phase names are present, not the prose dates.	The human approval step: the PM reads the plan before it is emailed.
KL-2	T12	Intermittent: seen in Run 3, absent in Runs 4 and 5 (Run 2 output truncated). The sprint update says "the project is on track, with no blocked items" while T024 is Blocked on the board (Sprint 5). The claim is true for Sprint 2 but overstated.	Low to Medium (tone and overclaim)	Score only checks that the right Sprint 2 tasks appear and that no Sprint 3+ tasks do.	The human approval step. Also, the update is scoped to Sprint 2 by design.
KL-3	T8	T006 (In Progress, due 2026-04-21) is not listed. It falls under Watch per the rule (In Progress due within 7 days), not At risk.	Low (rule-consistent, but a reader might expect it)	The expected list follows the at-risk rule, so the test passes.	Document the Watch category. Optionally add it to the prompt later.
KL-4	T11	Run 2 listed the 2026-05-05 milestone inside the 14-day window, which was wrong. The check was a false pass. It was fixed with a stricter check and a prompt rule in Run 3.	Resolved	The original check only looked for the two required deliverables.	Fixed in Run 3. Keep the Run 2 row as evidence.
KL-5	All	Scoring is by code heuristics (IDs, counts, route, required phrases). It cannot judge the accuracy of the prose, which is how KL-1 and KL-2 got through.	Design limitation	n/a	Risk relevance is judged by an LLM. Everything else relies on human review before sending.

ID	Test	Finding	Severity	Why the check missed it	Mitigation in place
KL-6	T3, T7	The judge scores register fit, not reasoning quality. Run 5 T3 scored 5 although the R04 justification cites a task that does not exist ("implied but not explicitly listed"). Run 4 T3 linked R06 (Cost Overruns) to the blocked security task T024, which relates to security (R09), not cost, yet the judge still scored it 5.	Low to Medium (weak justifications pass)	The judge rubric and the unsupported-items check look at risk IDs and categories, not at the task links in the "why" text.	The human approval step. Optional future check: verify every task ID in the justification against the task board.
KL-7	All	Model comparison not run. gpt-4.1-mini is blocked on the project's OpenAI allow-list (403), in both n8n and Langfuse. The writer, the code-scored judge and the Langfuse evaluator all run on gpt-4o-mini, so the judge is the same model family as the writer (self-preference risk).	Design limitation	Not a test failure. A platform access restriction.	The calibration test (fake R11 scored 2 after rule 8) and the human approval step. Next step: ask the project owner to allow a second model, or add a second provider (it needs a BRD line, since OpenAI is the approved provider).
KL-8	T3, T7 (risk judge)	Human-LLM calibration of the risk judge. Baseline, Judge v1, 14 items: 64% pass/fail agreement, kappa 0.26 (fair), 3 false passes, 2 false fails. Test set, 19 items: v1 58%, kappa 0.23, 7 false passes, 1 false fail. v2 68%, kappa 0.27, 3 false passes, 3 false fails. The judge catches register-fit defects but passes IDs-only answers, generic reasons and wrong task links.	Medium (judge validated for register fit only)	Earlier runs were 12/12 on good answers, and a lenient judge looks identical to a good one.	Human approval step. Judge v2 evaluated offline and recommended, not yet deployed in Mira Eval; Runs 2 to 5 used v1. Hybrid code checks and fine-tuning are listed as future work.

ID	Test	Finding	Severity	Why the check missed it	Mitigation in place
KL-9	T3, T7 (risk judge)	Judge v2 rule 9 misfires: it lists real task IDs (for example T006, T009, T024) as unsupported and fails good answers (K07, L03, L05). Rule 10 (IDs only) had no effect: K08 scored 5 in both versions.	Low to Medium (causes extra human review, blocks nothing)	The judge has no copy of the task board, and the model does not apply rules 9 and 10 reliably.	Task-ID and completeness checks belong in code (planned). The human approval step already reviews every answer.
KL-10	Approved email	Underscores in file names were read as italics and stripped (`project_description.txt` became `projectdescription.txt`). The draft email and approval form still show raw markdown.	Low (cosmetic)	Code checks never read the sent email.	Fixed 2026-10-22 for the approved email by enabling Middle Word Underscores in the To HTML node; draft formatting left as is. Logged as KL-10 in the Known Limitations sheet.

Root-cause themes

Three patterns explain most of the failures.

1. **Checks look at structure, not meaning.** The scores are code heuristics (KL-5): the T1 and T12 checks confirm the right phases and tasks appear. They cannot see a wrong date or an overclaim in the prose (KL-1, KL-2).
2. **The model fills gaps with plausible detail.** gpt-4o-mini cites tasks that are not on the board, such as T027 and T028 in K02, and gives wrong reasons for real risks (K01). The same behaviour produces the wrong dates in KL-1.
3. **The judge shares the writer's blind spots.** It is the same model family as the writer and scores register fit more than correctness (KL-6, KL-7, KL-8). Prompt rules help on some items and misfire on others (KL-9).

A fourth, smaller cause is plumbing: a formatting step in the send path altered file names (KL-10).

LLM judge reliability

The risk-relevance judge agrees with a human on pass/fail only 58% to 68% of the time, which is fair at best (kappa 0.21 to 0.40), so it supports a human reviewer and does not replace one.

Human ratings were done blind on 33 answers: a 14-item baseline (judged by v1) and a 19-item test set (K01 to K13 plus L01 to L06), which both v1 and v2 judged. Before that, a fake-risk check in Run 5 tested the judge's grounding rule: an invented risk (R11) scored 3 and 4 before rule 8 and 2 and 2 after, while real answers stayed at 5. Both judge versions use the same rubric; v2 adds three rules for task IDs, IDs without mitigation, and generic reasons.

Set	Judge	Items	Agreement	False passes	False fails	Kappa
Baseline	v1	14	64.29%	3	2	0.26
Test	v1	19	57.89%	7	1	0.23
Test	v2	19	68.42%	3	3	0.27

Three caveats. First, v2 was diagnosed on the same 19 items it is scored on, so its figures are optimistic. Second, all three v2 false fails come from rule 9 calling valid task IDs out of range, and two of its correct fails (K03, K05) rest on the same wrong reason. Third, false passes are the costly error here, because they let unsupported risks through, and v1 produced seven of them.

Mitigations and fixes

A human approves every plan, risk assessment and status report before it is emailed, and that is the main safeguard against the failures above.

- **Human approval (Gmail):** the PM approves or rejects with a reason. Decisions are timestamped (for example 2026-10-23T02:50:27Z) and form an audit trail.
- **Clarify routes skip approval by design.** They only ask for missing detail, so they send no generated claims.
- **Fixed:** T11 (stricter date check plus prompt rule, Run 3) and KL-10 (Middle Word Underscores, verified in the Oct 22 approved email against the Oct 20 email as the before case).
- **Measured, not fixed:** judge v2 improves agreement from 58% to 68% on the test set and is recommended, but is not deployed.

- **Cost control:** about $0.032 per 100 requests on gpt-4o-mini, tracked per generation in Langfuse.

Residual risk and future work

The main residual risk is a wrong date or overclaim that passes every automated check and reaches a client if the reviewer misses it.

- **Hybrid checks:** keep the judge for register fit, and add code checks for grounding (task and risk IDs checked against the board and register, completeness, dates matched to the board).
- **Judge v3:** give the judge the full 25-task board so it can verify reasons, and retest on a fresh set, since the current test set is spent.
- **Model comparison:** run the second-judge comparison once the allow-list permits it (KL-7).
- **Fine-tuning:** fine-tuning a judge or writer on rated examples would likely help most, but it was too expensive for this project and is left as future work.
- **Human annotation in Langfuse:** annotation queues with a score config are the natural production extension. Here, calibration was done offline in the sheet on 33 rated items.